Test Report — Waggle & Populace

Everything that was run against both products, what it found, and — set out just as plainly — what was never tested. Figures here are read from run artefacts and re-verified commands, not recalled. Where a number could not be sourced, it is marked as not measured rather than estimated.

15 August 2026	Buzz 1.5 (6)	Populace 0.1.0	Supabase / Postgres	Gigzen
Report date	Android, closed testing	Published on npm	Backend under test	Author

> **Since this report.** Figures here are as measured on 15 August 2026 and are not updated in place. As of 29 August: Populace is **1.3.0** (was 0.1.0) and its suite is **101 self-tests** (was 78); the app is **Waggle 1.0** under `com.gigzen.waggle` (was Buzz 1.5 under `com.masayaako.driver`) and ships **43 languages** and **70 currencies**; the shared database is **18 tables under 44 RLS policies**. Nothing below has been re-run, so nothing below has been changed.

> **As of 4 September 2026.** The shared database is **20 tables under 49 row-level-security policies** — the two added tables carry stories, which expire twenty-four hours after posting by a rule in the read policy rather than by a client remembering to filter. the engine is **1.3.1** on npm — the note above says 1.3.0, which was already a patch behind on the day it was written — and Populace Studio is **1.0.13** across **twelve releases**, each published with its SHA-256. Waggle is an **8.2 MB** signed build that now keeps recording with the screen off, through a native Android foreground service verified on a handset. Two defects found since: currency resolved from the handset's UI language rather than the driver's location, so a driver in Mumbai on an en-US phone was quoted dollars under a setting that says "by location"; and a distance filter that ignored accuracy, so an imprecise fix could pay a parked driver. Neither was caught by a test. Both were found by opening the app and looking at it.

> **As of 12 September 2026.** The engine is **1.3.2** and Populace Studio is **1.0.14**, which is light rather than dark. The self-test suite is **119 checks** — the note above says 101. Most of the new ones cover a defect in the part of Populace that matters most: **the guard that refuses to run against production did not work**. It compared the target against the forbidden list as whole strings, scheme included, so a production URL written over `http://`, as a Postgres connection string, as a `wss://` realtime URL, with credentials in front of the host, or as any subdomain went straight past it. Five realistic spellings, and the one check that exists to stop simulated people reaching a real database stopped none of them. It is fixed — hosts are compared as hosts now — but the reason it survived is the part worth saying: that guard had **no test coverage at all**, while every other safety property in the file had some. It was found by testing the thing that had never been tested, not by the suite.

> **As of 13 September 2026.** Engine **1.3.3**, Studio **1.0.15**, **121 checks**. **Most "13/13" figures below overstate coverage.** Until this release, coverage counted the methods an adapter implemented, not the methods a run called. `refreshSession` fires every thirty minutes, so only a run longer than that can reach it. The thirty-minute, thirty-user run is the one that did: all thirty sessions renewed, and its 13 of 13 is real. The twenty-, ten- and eight-user runs and the re-run after the fixes each exercised **12 of 13** — their tables have no `refreshSession` row. The Gitea run and the regression run have no per-method table, so their exercised figure cannot be restated; read their 13/13 as methods implemented, not methods tested. Found by the first run against a hosted target, which reported 13/13 after five minutes. Coverage now counts what ran.

> **As of 16 September 2026.** Engine **1.3.4**, **121 checks**. **Nothing on this page changes.** 1.3.4 renames the reference adapter from `buzzbuzz` to `waggle` and does nothing else: the checks, how coverage is counted and every run below are unaffected, and so is a Studio build that bundles it. The runs themselves keep the adapter name they really used.

1 · Headline

300	0	9	78
concurrent users, sustained	API failures across 3.17 million calls	real bugs found across both products	Populace self-tests, all passing

A note on the name: the app was called *Buzz Buzz* while every run in this document was carried out, was renamed to **Buzz** on 16 August 2026, and again to **Waggle** on 27 August 2026 — at which point its package also changed from `com.masayaako.driver` to `com.gigzen.waggle`. The product, the code and the figures are unchanged; only the name is. Run artefacts, version rows and commit messages from before each change still carry the name in use at the time, and are left that way on purpose: a test report that rewrites what it called things is a test report you cannot check.

The single most useful result in this document is not the clean run. It is that a **finished, signed, manually tested application could not create a working account**, and that this was found in the first fifteen seconds by software rather than by a person.

The second most useful is that the **sixth defect was found on the sign-off run** — the run whose whole purpose was to confirm there was nothing left to find.

2 · How the products were tested

Waggle is a multi-user product: a social feed, group chat, presence on a map, and permission rules deciding who may read what. None of that can be proven by one developer with one account, however carefully they tap through every screen. So the primary method was to generate a population and let it use the app through its real API, as genuinely authenticated users, with the app's own permission rules applying.

METHOD	WHAT IT COVERS	WHO RAN IT
Populace simulation	Concurrent multi-user behaviour against the live backend	Automated
Populace self-tests	The testing tool's own correctness	Automated
TypeScript compilation	Type safety across the app source	Automated
Release build	That the shipped artefact builds and is signed	Automated
Browser instrumentation	Contrast, overflow, console errors on all web properties	Automated
Screen walkthrough	Every screen against the live backend	Manual
Two- and four-device paths	Notification delivery on real hardware	**Not completed — see §8**

3 · Populace against Buzz — 9 August 2026

Six simulated drivers, three in Manila and three in Mumbai, against the live backend with 17 tables and 48 row-level-security policies. Each signed up for a real account, set a profile, drove a plausible route, posted, read the feed, liked and commented, opened conversations, sent messages and joined a group. Nobody told Populace where the bugs were.

3.1 The first run — five defects in three and a half minutes

The application had already passed a full manual test of every screen by the person who wrote it. The run found the following:

#	DEFECT	EFFECT	ROOT CAUSE
1	Signup created no profile row	**Every new account broken**	Upsert touching a deliberately unreadable column
2	Likes rejected at random	**~1 tap in 4 failed**	Upsert on a table with no UPDATE policy
3	Profile edit failed silently	**Changes discarded**	Same upsert, same column
4	RPC argument named wrongly	**Call rejected**	Populace's own reference adapter
5	A write's error was ignored	**Report blamed the wrong method**	Populace's own reference adapter
6	Re-joining a group you were already in	**"Join" failed silently**	Upsert on a table with no UPDATE policy — found later, see §3.3

> **The rule worth keeping**
>
> **You cannot upsert a column you cannot select.** In Postgres, `INSERT … ON CONFLICT DO UPDATE` requires SELECT permission on every column it touches. Three of the five defects were this same mistake wearing different clothes. It is invisible in code review, invisible in a single-user walkthrough, and obvious within seconds to six accounts signing up at once. It surfaced a third time in defect six, long after the first two were fixed, which is the real lesson: not a bug to fix once, but a habit to break.

Defects 4 and 5 were in our own testing tool, and 5 is the one worth dwelling on: an unchecked error is the precise fault Populace exists to catch, and it was sitting in our code. Because the failure was swallowed at signup, the report blamed a later method — it said *post failed 14 times* when the truth was *the profile row was never created*. Making that one line throw turned three confusing symptoms into one accurate sentence.

3.2 The re-run after the fixes

All five were fixed and the run repeated against the same backend. Every implemented method was exercised; every account created was deleted afterwards.

METHOD	CALLS	FAILURES	P50	P95
reportLocation	172	**0**	414 ms	571 ms
recentPostsByOthers	61	**0**	112 ms	129 ms
like	59	**0**	108 ms	133 ms
post	22	**0**	109 ms	128 ms
comment	22	**0**	114 ms	141 ms
openConversation	21	**0**	115 ms	142 ms
sendMessage	21	**0**	113 ms	142 ms
createUser	6	**0**	436 ms	1.6 s
setProfile	6	**0**	121 ms	305 ms
deleteUser	6	**0**	130 ms	293 ms
listGroups	2	**0**	—	—
joinGroup	2	**0**	—	—
Total	**400**	**0**	—	—

Coverage 13/13 methods. Six accounts created, six deleted, nothing left behind. Simulated activity: 6.2 km driven, 22 posts, 59 likes, 22 comments, 21 messages, 2 group joins. Account deletion is included deliberately — it is the path almost nobody exercises and the one regulators ask about.

3.3 Where the capacity ends — 6 to 36 concurrent users

Six runs at increasing concurrency, all of them clean. The zero-failure results are the least interesting thing about them: taken together they show **where this backend starts to queue**, which is the number worth knowing before real riders arrive rather than after.

USERS	CALLS	FAILURES	`REPORTLOCATION` P50	P95
6	400	**0**	414 ms	571 ms
8	1,626	**0**	727 ms	935 ms
10	2,030	**0**	669 ms	1.0 s
20	6,450	**0**	645 ms	1.1 s
30	6,539	**0**	799 ms	2.8 s
36	**7,485**	**0**	**757 ms**	**1.4 s**
30 local	**18,372**	**0**	**51 ms**	**88 ms**

Every row counts what that many users actually did, and the 36-user row is no exception: **7,485 calls across twelve methods, zero failures**. It reached 36 by way of a 50-user run — fourteen accounts were refused at signup, before the run began, and the run proceeded with the 36 that existed. Those fourteen were provisioning, not load, and they are reported as their own finding below rather than charged to this row. No call failed at 36 concurrent users.

One row needs a pointer: **the ten-user figure is the regression re-run of §3.6** (2,030 calls), not the original ten-user run, which made 2,116 calls and has its own detail block below. Both were clean; the later one is quoted here because it ran against the current engine.

Correction: the 30-user tail was an outlier

An earlier version of this section read the p95 jump at thirty users — 1.1 s to 2.8 s — as the write path beginning to saturate. **A later run at higher load contradicts it.** At thirty-six concurrent users p95 came back to 1.4 s, so the 2.8 s did not reproduce under more pressure and cannot be load.

That claim is withdrawn. It is the second latency finding this report has had to retract, and the lesson is the same both times: **a single measurement between runs on different days is not a trend**, however good a story it tells. The paragraph below is kept as a record of the reasoning that was wrong, because deleting it would hide that this happened twice.

What actually limits this system: signup throughput

A run of fifty agents found the real ceiling, and it is not in Waggle. **Fourteen of fifty accounts could not be created**, every one of them refused by Supabase Auth with `Request rate limit reached` — a free-tier quota on signups per hour, hit by creating fifty accounts in seconds.

Verified as external rather than assumed: **there is no rate-limiting code anywhere in the application or its SQL**, and the message is passed through verbatim from the auth provider.

Two things follow. Real riders sign up over weeks, so this would not occur naturally — but it *would* occur during a bulk onboarding or a launch event, and it is a plan limit rather than a wall, so a paid tier raises it. And it says nothing about how many people can *use* the app at once: the ceiling found here is on account creation, and the thirty-six that got through ran for ten minutes with no failures at all.

The honest limit statement: Waggle has been shown to sustain 36 concurrent users cleanly. That is the highest tested, not a proven maximum — nothing yet says where its own ceiling is.

The reasoning that was withdrawn

From six to twenty users the median held between 414 and 727 ms with no trend — the variation there is noise between runs on different days, and an earlier version of this report was wrong to read the six-to-eight step as superlinear growth.

At thirty something changes. The median moves modestly, 645 to 799 ms, but **p95 goes from 1.1 s to 2.8 s** — a 2.5× jump. `createUser` p95 reaches 2.1 s and `setProfile` 1.1 s. Every read stays flat near 240 ms.

That shape — median steady, tail stretching, and only on the write-heavy calls — is what a resource beginning to saturate looks like. Most requests are still served immediately while a growing minority queue behind something. It is concentrated in `reportLocation`, which alone is 2,793 of the 6,539 calls, and in the signup path.

Read this as one measurement, not a capacity model. It is a single run on a free-tier project, and this report has already had to withdraw one latency claim that turned out to be run-to-run variance. What makes this one worth recording is that it has a coherent story: the calls that slow down are the ones that write, and the divergence between median and tail is a pattern rather than a lone number. The honest summary is that **somewhere between twenty and thirty concurrent writers, this project's write path starts queueing** — comfortably beyond closed testing, and worth measuring deliberately before it matters.

Run detail · thirty concurrent users, thirty minutes

The longest and largest run in this document, and the only one to exercise every method the adapter has. Thirty agents for 1,856.7 seconds — just under thirty-one minutes — against a **local Supabase stack** built that evening from this project's own SQL, including the two migrations added the same day. Read the caveat below before comparing its latencies with anything else here.

METHOD	CALLS	FAILURES	P50	P95
reportLocation	8,033	0	51 ms	88 ms
recentPostsByOthers	2,693	0	11 ms	21 ms
like	2,693	0	8 ms	15 ms
post	1,183	0	10 ms	20 ms
openConversation	1,126	0	8 ms	18 ms
sendMessage	1,126	0	8 ms	18 ms
comment	1,054	0	7 ms	13 ms
listGroups	172	0	7 ms	17 ms

joinGroup	172	0	7 ms	18 ms
createUser	30	0	154 ms	189 ms
setProfile	30	0	15 ms	21 ms
`refreshSession`	30	0	252 ms	308 ms
deleteUser	30	0	16 ms	32 ms
Total	**18,372**	0	—	—

332.7 km driven · 1,183 posts · 2,693 likes · 1,054 comments · 1,126 messages · 172 group joins. **0 API failures, 0 transport failures, 0 retries, 0 outages, 0 engine errors**, and **30 accounts created and 30 removed**, cleanup verified empty afterwards. This is the first run long enough for `refreshSession` to fire for every agent: all thirty sessions were renewed mid-run and none dropped, which is why coverage reads 13 of 13 here rather than 12.

> **Do not compare these latencies with the cloud runs**
>
> Seven to eleven milliseconds is a loopback socket, not a network. The same twelve methods measured 176–180 ms against the cloud project earlier the same evening, and that gap is the internet, not the code.
>
> What a local run does establish is the part a cloud run cannot separate: **with the network removed, thirty concurrent users produced no failure of any kind for half an hour**. It also ran against a schema built that day from the project's own SQL files, so the migrations are not merely syntactically valid — a full workload runs on top of them.
>
> The attempt to repeat this run against the cloud project is recorded in §3.8. It did not finish, and the reason was not Waggle.

Run detail · thirty-six concurrent users, ten minutes

The largest run in this document, and the cleanest. It was launched as a fifty-agent run; fifty accounts could not be created, so thirty-six went to work instead, for 931.3 seconds against the isolated cloud test project. Provisioning and the run are reported separately below, because they failed and passed for entirely different reasons.

Provisioning, before the run. `createUser` was called 50 times: **36 accounts created, 14 refused** by Supabase Auth with `Request rate limit reached` (p50 653 ms, p95 1.3 s across all fifty attempts). This is the signup ceiling described above — a free-tier quota on account creation, not a property of Waggle. The run then began with the 36 accounts that existed.

METHOD	CALLS	FAILURES	P50	P95
reportLocation	3,240	0	757 ms	1.4 s
recentPostsByOthers	1,119	0	246 ms	410 ms
like	1,117	0	237 ms	307 ms
post	474	0	243 ms	472 ms
comment	441	0	237 ms	357 ms
openConversation	423	0	242 ms	465 ms
sendMessage	423	0	243 ms	390 ms
listGroups	70	0	241 ms	532 ms
joinGroup	70	0	242 ms	498 ms
setProfile	36	0	275 ms	571 ms
deleteUser	36	0	276 ms	464 ms
Total, in run	**7,449**	0	—	—

With the 36 successful `createUser` calls added, **7,485 successful calls, none of which failed**. Every method the app has was exercised: eleven here plus `createUser`, and `listConversations` covered by the smoke check beforehand.

Throughput held for the full ten minutes. The engine reports every 24th tick, and the interesting thing is what does *not* happen — each quarter of the run does roughly the same amount of work as the one before it, so nothing sagged as sessions aged and tables grew:

TICK	DISTANCE	POSTS	LIKES	COMMENTS	MESSAGES	KM THIS QUARTER
24 / 120	28.2 km	109	234	90	90	28.2

48 / 120	55.6 km	194	475	206	178	27.4
72 / 120	80.1 km	278	679	277	258	24.5
96 / 120	106.2 km	374	892	360	356	26.1
120 / 120	**132.5 km**	**474**	**1,117**	**441**	**423**	26.3

132.5 km driven · 474 posts · 1,117 likes · 441 comments · 423 messages · 70 group joins. **0 API failures, 0 transport failures, 0 retries, 0 outages, 0 engine errors**, and **36 accounts created and 36 removed** — cleanup verified empty afterwards. Populace exited non-zero on this run, correctly: it was asked for fifty users and delivered thirty-six, and a tool that reported success there would be worthless. *The run itself did not fail.*

Run detail · thirty concurrent users, ten minutes

Thirty agents for 943.1 seconds against the isolated cloud test project. All thirty accounts were created without trouble — the signup quota was not reached at this size — and the run returned `exit 0`.

METHOD	CALLS	FAILURES	P50	P95
reportLocation	2,793	**0**	799 ms	2.8 s
recentPostsByOthers	984	**0**	241 ms	387 ms
like	984	**0**	236 ms	330 ms
post	403	**0**	238 ms	401 ms
comment	397	**0**	240 ms	416 ms
openConversation	389	**0**	240 ms	381 ms
sendMessage	389	**0**	243 ms	395 ms
listGroups	55	**0**	242 ms	506 ms
joinGroup	55	**0**	241 ms	522 ms
createUser	30	**0**	701 ms	2.1 s
setProfile	30	**0**	254 ms	1.1 s
deleteUser	30	**0**	302 ms	464 ms
Total	**6,539**	**0**	—	—

116.2 km · 403 posts · 984 likes · 397 comments · 389 messages · 55 group joins. Verdict `clean`: 0 API failures, 0 transport failures, 0 retries, 0 outages, 0 engine errors, 30 accounts created and 30 removed. The 2.8 s `reportLocation` tail in the first row is the measurement that produced the withdrawn saturation claim — it did not reproduce at thirty-six users, where the same figure came back to 1.4 s.

Run detail · twenty concurrent users, fifteen minutes

Twenty agents for 1,264 seconds against the isolated cloud test project, preceded by `clean`, `doctor` and `smoke`, and followed by `clean` again. At 21 minutes it is the longest run in this document.

METHOD	CALLS	FAILURES	P50	P95
reportLocation	2,810	**0**	645 ms	1.1 s
recentPostsByOthers	931	**0**	236 ms	311 ms
like	930	**0**	233 ms	323 ms
post	437	**0**	236 ms	384 ms
openConversation	394	**0**	235 ms	369 ms
sendMessage	394	**0**	236 ms	338 ms
comment	384	**0**	234 ms	335 ms
listGroups	55	**0**	236 ms	379 ms
joinGroup	55	**0**	237 ms	375 ms
createUser	20	**0**	724 ms	1.3 s

setProfile	20	**0**	318 ms	625 ms
deleteUser	20	**0**	274 ms	389 ms
Total	**6,450**	**0**	—	—

116.3 km · 437 posts · 930 likes · 384 comments · 394 messages · 55 group joins. Verdict `clean`: 0 API failures, 0 transport failures, 0 retries, 0 outages, 0 engine errors, 13/13 methods, 20 accounts created and 20 removed. At 21 minutes this is also the first run long enough to exercise session refresh properly — all twenty sessions survived it.

Run detail · ten concurrent users, ten minutes

Ten agents across Manila and Mumbai for 809.1 seconds against the isolated cloud test project. `clean` was run first to prove the target was empty, then `doctor` and `smoke`, then the run, then `clean` again.

METHOD	CALLS	FAILURES	P50	P95
reportLocation	953	**0**	669 ms	912 ms
recentPostsByOthers	287	**0**	234 ms	284 ms
like	283	**0**	237 ms	345 ms
post	148	**0**	234 ms	274 ms
comment	127	**0**	237 ms	320 ms
openConversation	121	**0**	237 ms	289 ms
sendMessage	121	**0**	241 ms	338 ms
listGroups	23	**0**	240 ms	340 ms
joinGroup	23	**0**	234 ms	329 ms
createUser	10	**0**	655 ms	1.6 s
setProfile	10	**0**	266 ms	799 ms
deleteUser	10	**0**	320 ms	598 ms
Total	**2,116**	**0**	—	—

39.7 km · 148 posts · 283 likes · 127 comments · 121 messages · 23 group joins. Verdict `clean`: 0 API failures, 0 transport failures, 0 retries, 0 engine errors, 13/13 methods, 10 accounts created and 10 removed with none stranded.

Run detail · eight concurrent users, ten minutes

The only run at eight agents. Against the isolated cloud test project, not production: `doctor` confirmed *2 production host(s) denied* before it started.

METHOD	CALLS	FAILURES	P50	P95
reportLocation	727	**0**	727 ms	935 ms
recentPostsByOthers	234	**0**	235 ms	291 ms
like	233	**0**	240 ms	337 ms
openConversation	106	**0**	239 ms	445 ms
sendMessage	106	**0**	248 ms	385 ms
comment	84	**0**	246 ms	329 ms
post	78	**0**	233 ms	290 ms
listGroups	17	**0**	249 ms	329 ms
joinGroup	17	**0**	280 ms	351 ms
createUser	8	**0**	676 ms	1.5 s
setProfile	8	**0**	239 ms	750 ms

deleteUser	8	0	320 ms	577 ms
Total	**1,626**	0	—	—

806.8 s · 27.8 km · 78 posts · 233 likes · 84 comments · 106 messages · 17 group joins. Verdict `clean`: 0 API failures, 0 transport failures, 0 retries, 0 engine errors, 13/13 methods covered, 8 accounts created and 8 removed with none stranded. First run in which `openConversation` and `sendMessage` were exercised at volume.

3.4 The local runs, and a sixth defect

Work continued against a **local Supabase stack** rather than the cloud project, which takes the network out of the measurement. Two further runs, six agents for about five minutes each:

RUN	CALLS	FAILURES	DURATION	ACTIVITY
Local, first	627	0	308.3 s	10.6 km · 43 posts · 87 likes · 37 comments · 35 messages · 6 joins
Local, sign-off	**672**	0	307.9 s	10.7 km · 29 posts · 104 likes · 36 comments · 43 messages · 3 joins

Both reported `preclean=0 · run=0 · postclean=0` — clean before, clean during, clean after. The sign-off run was the **first `exit 0` Populace had ever returned against Waggle**.

> **Defect six, found on the sign-off run**
>
> The run whose purpose was to confirm the app was finished did not confirm it. `joinGroup` used `.upsert()`, which compiles to `ON CONFLICT DO UPDATE`, and `group_members` has no `UPDATE` policy. **Tapping "Join" on a group you had already joined failed.**
>
> Third appearance of the same upsert family, after signup and likes. Corrected in both the app and the reference adapter, then the run repeated — the 672 calls above.

> **Do not compare these latencies with §3.2**
>
> The local runs put `reportLocation` at a p50 of **19–21 ms** against **414 ms** on the cloud project. That is the network being absent, not the software being twenty times faster. Local figures are good for finding logic and permission faults; only the cloud figures say anything about what a rider would experience.

> **What these runs do not claim**
>
> Six users for three to five minutes is a **correctness run, not a load test**, and each ran against a project that was effectively empty. The latencies are what six concurrent users saw and nothing more. No conclusion about behaviour at scale follows.

3.5 · A second backend, and what it exposed

Every run above uses the same backend technology. That made the strongest caveat in earlier versions of this report an open one: **Populace had been pointed at exactly one real application, and that application was ours.** A testing tool that has only tested its author's own app has been shown to work once, not shown to be portable.

So a second backend was built, deliberately unlike the first on every axis that could hide an assumption, and shipped in the repository as `examples/rest-api/` so the result can be reproduced by anyone:

	WAGGLE	THE REST DEMO
Transport	supabase-js client	plain `fetch`
Identifiers	UUID strings	integers
Errors	`{data, error}` tuples	HTTP status + `{error}`
Auth	session on a client object	bearer token in a header
List endpoints	row objects	bare id arrays
Enforcement	row-level security, in the database	application code, in the handler

Result: no failures across 430 API calls, 13/13 methods, 6 accounts created and 6 removed. The engine could not tell the two apart.

Two defects in Populace, found by pointing it somewhere new

The engine could not read a bare identifier. It read `target.id` from whatever the feed returned, so an API answering `[1, 2, 3]` produced `undefined` on every like and join. `smoke` had always accepted both shapes — so an adapter like this **passed the smoke test and then failed on the first tick of a real run**, which is the exact case smoke exists to rule out.

`smoke` called `createUser` with a different object than the engine. The contract documents `{name, phone, persona, index}`; smoke passed a flat persona carrying a password the engine never supplies. An adapter written against either shape passed one check and failed the other. This was only visible because a run and a smoke test disagreed about the same new backend.

Both are now pinned by self-tests, including one that fails if the two ever drift apart again. The bare-identifier test was verified to fail with its fix reverted: a regression test that cannot fail proves nothing.

3.6 Regression run after those changes

The 2,116-call result above was measured on code that has since changed. Waggle was therefore re-run in full against the modified engine: **2,030 API calls, 0 failures**, 10 agents, 818.1 s, 13/13 methods, 10 accounts created and 10 removed, verdict `clean`. Fixing portability did not cost correctness on the original backend.

3.8 The same run over the network, which did not finish

The thirty-user, thirty-minute run above was repeated against the cloud test project the same evening, deliberately, to get the figure a local stack cannot give. **It did not complete.** It is recorded here because a report that only keeps the runs that worked is not a record.

PROPERTY	LOCAL	CLOUD, SAME WORKLOAD
Completed	**30 of 30 min**	**10.5 of 30 min**
API calls	18,372	1,569
Failures	**0**	**214, all transport**
`reportLocation` p50	51 ms	550 ms
Accounts removed	**30 of 30**	**0 of 30**

Every failure was transport: 149 calls refused once the breaker opened, 30 `reportLocation` timeouts at 20 s, and a handful of `fetch failed`. Every method that reached the server was clean — `like` 179, `sendMessage` 85, `comment` 72, `createUser` 30, all at zero.

The cause was DNS, and it was measured rather than assumed

The obvious story is a slow connection. It was not. Breaking a single request into phases puts the time somewhere specific:

```
dns=11.06s  tcp=11.13s  tls=15.44s  total=17.60s
```

Name resolution took eleven seconds; the TCP connection completed seventy milliseconds after it. The path to the server was fine — finding its address was not. The machine was on a phone hotspot whose primary resolver is an IPv6 link-local address that times out before falling back, and queries to 8.8.8.8 and 1.1.1.1 were blocked outright, so the only working resolver was the hotspot's own at roughly 2.2 seconds a lookup. Over the same period `github.com` would not resolve at all.

This is why the local comparison matters. Identical code, identical workload, thirty agents: zero failures for half an hour with no name lookups in the path, and a collapse inside eleven minutes with them. **Nothing in this section is evidence about Waggle.**

Defect eight, in Populace: cleanup cannot outlive the process

The failed run left **thirty simulated accounts live in the test project**, and a follow-up `clean` could not remove them either.

The circuit breaker resets before teardown precisely so a dead-network verdict cannot block cleanup — there is a comment in `net.mjs` explaining why, because an earlier version left five invented accounts behind. That safeguard ran correctly and was still not enough: each driver hit twelve consecutive failures on the retry path.

The real weakness is narrower than "the network was bad". **The identities exist only in the run's memory.** Once the process exits, `clean` has to re-derive them and can fail in exactly the same way. Writing each created identity to disk as it is made would let a later cleanup finish with the network needed only to perform the deletion, not to know what to delete. Not yet implemented.

3.9 · The 21 August runs — thirty to three hundred drivers

Seven counted runs in one day against a local Waggle backend, from thirty simulated drivers to three hundred, across thirty cities in seventeen writing systems. **3,174,576 API calls and zero API failures.** 740 accounts created, 740 removed, verified by querying the database afterwards rather than by trusting the tool's own report.

RUN	DRIVERS	CITIES	CALLS	API FAILS	NETWORK	VERDICT
Repeatability ×3	30	4	140,691	**0**	0	clean
Mega	50	5	232,837	**0**	0	clean
Hundred	100	10	467,158	**0**	0	clean
Two Hundred	**200**	**20**	**932,455**	**0**	**0**	**clean**
Three Hundred	300	30	1,401,435	**0**	1	inconclusive

The three-hundred run is marked inconclusive because one `post` call never reached the server — a socket exhausted on the test machine, after 1,316 retried attempts across the run had succeeded. Its API failure count is zero, like every other row. Populace will not label a run clean when it could not make every call, even when nothing in the app went wrong.

Throughput stops scaling around two hundred drivers

Three runs share an identical shape — double engagement, one-second ticks — and differ only in population, which makes them the one honest comparison. A hundred drivers sustained 178 calls per second; two hundred sustained 241, a gain of **1.36×**; three hundred sustained 272, a gain of **1.13×**.

The extra work did not vanish, it became waiting: `reportLocation` moved from 142 ms to 606 ms at the median across that range. Nothing failed and no percentile ran away, so this is **saturation, not collapse** — where Waggle actually breaks is still unfound.

It did not slow down as the data grew

Latency rising with concurrency is expected. Latency rising *with time* is the failure that quietly kills a social feed: quick when the tables are empty, painful a year later at the same user count.

Every long run was checked quarter by quarter. In the three-hundred-driver run the per-tick cost **fell** — 3.06 s, then 2.96, then 2.59 — while the database grew past 122,000 posts and 260,000 likes. The last quarter ran against four times the data of the first and was the fastest of the four.

Two runs are excluded, and why

A five-hundred-driver run produced 106,110 calls and 51 `fetch failed` errors. The cause was this machine exhausting its 16,384 ephemeral TCP ports — at that rate it needed roughly 18,600 — not anything in the app. It measured the test harness, so it is not counted. **That is defect nine, and it is in Populace itself:** the server offers `Connection: keep-alive` and Node opens a fresh socket per sequential call.

A four-hundred-driver run was cancelled mid-flight for the same reason once ports passed 78% of the ceiling. Its 400 accounts were found and removed.

What governs every figure above: these runs are against a local Supabase stack over loopback, so no network is included. The same calls measured about 175 ms each against the hosted project before any load at all — a real driver's experience is this cost plus the network, not these numbers.

4 · Populace's own correctness

A testing tool that is wrong is worse than no testing tool, because it is believed. Populace therefore ships a suite that runs the whole product against an in-memory application with no backend of any kind.

```
$ node src/selftest.mjs
  ✔ without refresh, an expired session poisons the whole run
  ✔ refreshSession keeps a run alive past token expiry
  ✔ an expired run cannot even clean up after itself
  ✔ 0.88% failure rate is still a problem, not 'clean'
  ✔ the verdict line can never contradict the table beneath it
  ✔ a bug inside Populace is kept, not swallowed
  ✔ a run that broke internally is never reported clean
```

```
… 75 checks total

All checks passed — the engine is app-agnostic and the report is honest.
exit 0
```

Count verified by running the suite for this report, not taken from documentation: 75 passing checks, exit code 0.

4.1 The demo adapter run

Populace ships a demo application containing a deliberate row-level-security defect, so that a first-time user can watch the tool find something real. Run on 14 August 2026:

MEASURE	RESULT	READING
Verdict	**problems-found**	Correct — the seeded defect was found
API calls	308	Over 80 seconds, 6 agents
API failures	11 (3.57%)	All attributable to the seeded policy
Transport failures	**0**	Network was not a factor
Retries	0	No masking of intermittent faults
Agents signed in	**6 of 6**	No signup failures
Cleanup	**6 removed, 0 failed**	No orphaned accounts
Coverage	12/13	One method intentionally unimplemented

The separation of *API failures* from *transport failures* matters: it is what stops a flaky network being reported as a bug in your code.

5 · The shipped Android artefact

PROPERTY	VALUE	STATUS
Application ID	`com.masayaako.driver`	**Verified**
Version	1.5 (versionCode 6)	**Verified**
Variant	release	**Verified**
Size	8,273,594 bytes (7.89 MB)	**Measured**
SHA-256	`aead83377e32d57e49535997b857ac3fa6fa073c2689e36ffcc159f1779e35ec`	**Published**
Backend URL in bundle	production host; **0** occurrences of `127.0.0.1`	**Verified in APK and AAB**
Signature	APK Signature Scheme v2, certificate unchanged	`apksigner verify`
TypeScript	0 errors	**tsc --noEmit**
Minimum Android	8.0	**Verified**

A build that succeeded and shipped nothing

During this work a release build reported `BUILD SUCCESSFUL` while packaging **stale web assets**. The Capacitor sync step shells out to `pnpm`, which was not on the PATH; the step failed quietly and Gradle proceeded with whatever had last been copied. A green build is not evidence that your changes are in the artefact. A staleness gate was added so the build now fails rather than lies.

Defect seven: every artefact shipped today pointed at localhost

Found by the owner, not by this test suite, and only after the artefacts had been published. Signing in on a real handset returned `fetch failed`. The cause was in the build rather than the app: `.env.local` sets the backend to the local Docker stack for development, and **Vite resolves `.env.local` above `.env` in every mode, release builds included**. The APK on the download page, the AAB staged for Play and the copies on disk therefore all carried `http://127.0.0.1:54321` — an address that on a phone means the phone itself. None of them could ever have signed in.

The same file set `VITE_ENABLE_PUSH=false`, which compiled the push registration out of the bundle. **Part of why push had never been seen working on hardware is that it was not in the build being tested.**

Fixed with a `.env.production`, which Vite resolves above `.env.local`, so local settings can no longer reach a release build. Everything was rebuilt and then *checked* rather than trusted: the web bundle, the synced Android assets, the signed APK and the signed AAB each contain the production host and zero occurrences of the loopback address, and the APK is still v2-signed with the same certificate.

Two things this says plainly. **Section 3 tested the API, not the shipped app.** Populace drives the backend directly, so a client built against the wrong host is invisible to it — 7,485 clean calls proved nothing about whether the APK worked. And the staleness gate above did not catch this, because the assets were not stale: they were current, correct, and compiled against the wrong configuration.

6 · Internationalisation

MEASURE	RESULT	NOTE
Languages	**16, at full parity**	Including Arabic; right-to-left layouts included
Currencies	**27**	Selected from the rider's actual location
Countries	**49**	Extended so the published claim is true
Legal screens	**Translated**	Privacy and Terms in all 16

A defect worth recording: the bundled Inter typeface never rendered on Android. It was listed after `-apple-system` and `BlinkMacSystemFont` in the font stack, both of which resolve to Roboto on Android — so the font loaded on every device and was used on none of them. Ordering, not availability.

7 · Web properties

Three sites, each measured in a real browser rather than reviewed by eye. Contrast is computed from rendered colours against WCAG AA thresholds — 4.5:1 for normal text, 3:1 for large.

PROPERTY	LIVE	CONTRAST FAILURES	OVERFLOW	CONSOLE ERRORS
gigzen (company)	**HTTP 200**	**0**	**None**	**None**
populace	**HTTP 200**	**0**	**None**	**None**
buzz-buzz	**HTTP 200**	**0**	**None**	**None**

7.1 Defects the measurement found

Every one of these looked acceptable and was not:

- **White on the brand orange measured 3.32:1**, against the 4.5 required. Pure white on `#FF4D17` reaches only 3.35, so no text colour could have fixed it — the surface had to change. This defect was introduced twice, on two different sites, from the same instinct.
- **The light theme was unreachable** for any visitor whose operating system was set to dark. Palette values were defined in the base rule and overridden in a media query, but not in the explicit light selector, so choosing light kept the dark accents — light violet on a white page, at 2.28:1.
- **A near-black scrim survived its background.** After a section changed from black to a pale wash, an overlay still painted `rgba(5,7,8,.95)` over it: grey text on a black smudge.
- **Brand colours used as text.** Populace's lime measured 1.16:1 on paper and honey gold 2.1:1 on cream. Both are legitimate brand colours and neither is a text colour; each now has a darkened counterpart for type.
- **A muted grey at 4.40:1** — close enough to look fine and still fail.
- **A dangling custom property.** A renamed token left one gradient layer referencing a variable that no longer existed; that layer would have silently vanished.

Three findings during this work were defects in the measuring script rather than the pages — most notably that `color-mix()` computes to `color(srgb 1 1 1 / 0.93)`, whose 0-to-1 floats were being read as 0-to-255, making white register as near-black. They are recorded here because a measurement you have not questioned is not evidence.

8 · What was *not* tested

This section is the reason the rest of the report can be trusted. A test report that lists only passes is a marketing document.

AREA	STATUS	WHY
Two- and four-device paths	**Not completed**	Push notification delivery, and one person seeing another move on a map in real time, need real accounts on real handsets. Accounts cannot be created on the user's behalf, so these paths are documented as a manual script and remain unrun.
The shipped client, end to end	**Not tested**	Nothing in this document installs the APK and uses it. Populace speaks to the API directly, so everything in §3 would pass identically against a client that cannot start — which is exactly what defect seven turned out to be. Until someone installs the artefact and signs in, the app itself is unverified, however green the API numbers are.
Behaviour at scale	**Not tested**	The largest run was thirty-six concurrent users, and it was clean. That is the highest figure tested, **not a measured maximum** — the runs were stopped by a signup quota, not by the app slowing down, so where Waggle actually breaks is still unknown. These remain correctness runs at one measurement each. Nothing here supports a claim about hundreds or thousands of users.
Populace against other backends	**Not tested**	It now drives two unrelated backends (§3.5), so the engine is demonstrably not shaped around one stack. Both, however, are still ours. What remains unproven is how it behaves against an API designed by someone who never read the contract.
Google Play review	**Not passed**	The app has never been uploaded to Google Play, on any track. It has not been through review.
Company site dark theme	**20 known failures**	Dark is opt-in via an attribute and no control on the page reaches it. It is out of scope of the shipped design and is recorded rather than hidden.
Security penetration testing	**Not performed**	Permission rules were exercised by users with genuinely different identities, which is not the same as an adversarial review.
Real-world battery and GPS drift	**Not tested**	Simulated routes do not reproduce a phone in a pocket for eight hours.

9 · Conclusion

Both products do what is claimed of them, within limits this document states rather than hides. Waggle builds, type-checks, ships as a signed artefact, and survived a concurrent multi-user run against its live backend with no failures across 400 calls, and a thirty-six-user run of 7,485 calls with none either. Populace found six genuine defects in a finished application, passes 78 self-tests, and distinguishes a fault in your code from a fault in the network between you.

A seventh defect was found by the owner rather than by any of this, on a handset, after the artefacts were published: every build shipped that day was compiled against a developer's local backend and could not sign in at all. It is the most useful result in the document, because it marks the edge of what everything above actually measures. **Populace tests an API. It never opens the app.** A client built against the wrong host, with push compiled out, passes every server-side check ever written and still fails the moment a person taps “sign in”.

The honest summary is this: **what has been tested has been tested properly, and the largest untested areas are known and written down.** The most valuable outcome was not a passing run — it was discovering that an application which had passed a careful manual test by its own author could not create a working account, that three separate symptoms shared one cause, and that the same cause surfaced a third time on the very run intended to sign the work off.

> **Next**
>
> - Install the rebuilt APK and confirm sign-in works on a handset. Nothing else on this list matters until that is seen with human eyes.
> - Run the two- and four-device notification script on real handsets — now against a build that actually has push compiled in.
> - Point Populace at a backend that is not ours.
> - Decide the company site's dark theme: give it a control and fix it, or remove it.
> - Upload the first build to Google Play and complete review.

Prepared by Gigzen. Every figure in this report is taken from a run artefact or a command re-executed while writing it. Where something was not measured, it says so.